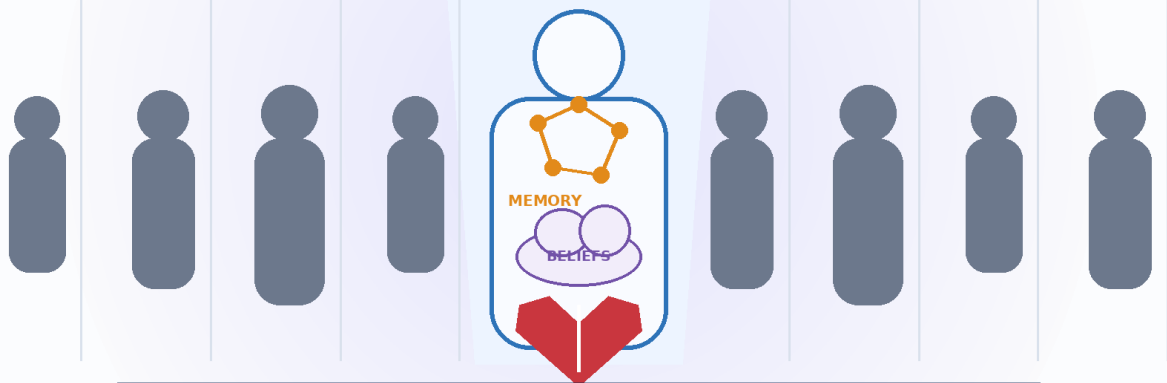


SONDER ENGINE

Systems Overview

An illustrated guide to persistent, epistemically constrained interactive fiction

Every Mind Has a Private World



Every mind knows only what it earned. The engine keeps private memories, beliefs, and motives separate while their observable consequences shape a shared world.

Sonder Engine treats every character as a private, fallible mind rather than a shared window into the whole world.

CORE IDEA

Objective truth, perception, memory, inference, belief, and narration are separate layers. The engine allows information to move forward only when a fictional mind has legitimately earned it.

How to Read This Guide

This document is a plain-language orientation to the engine's major systems. It explains what each subsystem owns, how a turn moves through the pipeline, and why the architecture produces more coherent long-form fiction than a single all-knowing prompt.

READING PATH

Start with the information barrier and agent map. The turn lifecycle explains runtime behavior; the later sections cover persistence, character psychology, memory, space, time, providers, and the database.

At a Glance

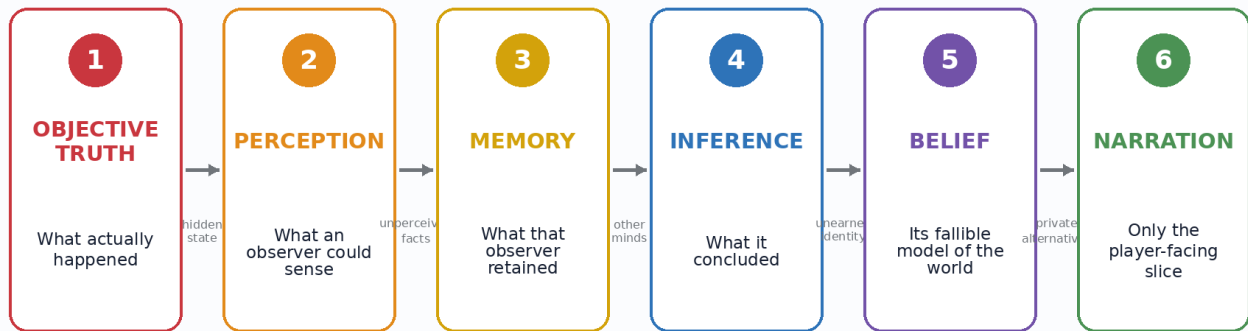
Area	Primary responsibility	Key modules
Information barrier	Prevents characters from using knowledge they did not perceive, learn, remember, or infer.	agents/perception.py, agents/character.py, agents/narration.py
Turn pipeline	Separates interpretation, perception, reaction, resolution, narration, and persistence.	agents/runtime.py, pipeline_context.py
Persistence	Stores inspectable variants and commits all primary effects atomically.	agents/storage.py, commit.py, db.py, checkpoints.py
Character mind	Maintains goals, affect, relationships, fallible models of others, and earned ruptures.	agents/character.py, affect.py, theory_of_mind.py
Memory and lore	Retrieves autobiographical memory and canon without leaking objective truth.	memory.py, agents/mapping.py
World and time	Tracks live scene truth, spatial access, frames, scheduled mechanics, and paradoxes.	scene.py, spatial.py, frames.py, paradox.py

1. What the Engine Is

Sonder Engine is a local, multi-agent interactive-fiction engine designed for persistent worlds and long-running roleplay. Its defining rule is an information barrier: a fictional mind may act only on knowledge it legitimately perceived, learned, remembered, or inferred.

Most roleplay systems collapse the whole world into one context window. Sonder instead separates six layers: objective truth, perception, memory, inference, belief, and narration. This makes mistaken beliefs possible, preserves secrets, and prevents an unrelated non-player character from mysteriously knowing the entire lorebook.

The Information Barrier



No fictional mind receives a layer to its left. Each transition removes knowledge that has not been earned.

Figure 1. Knowledge narrows as it moves from objective truth toward player-facing narration.

WHY IT MATTERS

The engine does not merely instruct a language model to avoid omniscience. It structurally withholds information that a particular role is not authorized to see.

2. The Cast of Agents

The engine is a pipeline of specialized agents with strict ownership boundaries. Role modules live in agents/ and may share common utilities, but only the runtime orchestrator knows the entire plan.

Agent	File	Owns	Must not
Director	agents/director.py	Objective causality: input interpretation, outcome resolution, world diffs, dice, obligations, and player-fact adjudication.	Character psychology, narration, or silent rewrites of the player's declared words.
Perception	agents/perception.py	A stateless filter that decides what each observer legitimately receives this beat.	Inventing intent, leaking hidden state, or granting unearned identity.
Character	agents/character.py; agents/loops.py	A private mind's declared behavior based on its own perception, memory, relationships, and interior state.	Deciding its own success or seeing objective truth and other minds.
Background	agents/background.py	At most one stateless, one-beat line for a named unregistered presence.	Persistent memory or psychology without promotion.

Agent	File	Owns	Must not
Narrator	agents/narration.py	Rendering the player-facing slice as prose.	Originating player conduct or revealing unperceived facts.
Mapping	agents/mapping.py	Routing lore, staging canon, and growing the world tree.	Deciding what a character perceives.
Commit	commit.py	The sole persistence boundary; validates provisional output and writes it.	Treating upstream model output as permanent before validation.
Runtime	agents/runtime.py	Plan construction, dispatch, streaming, cancellation, resume, reroll, and rerun-from-stage.	—

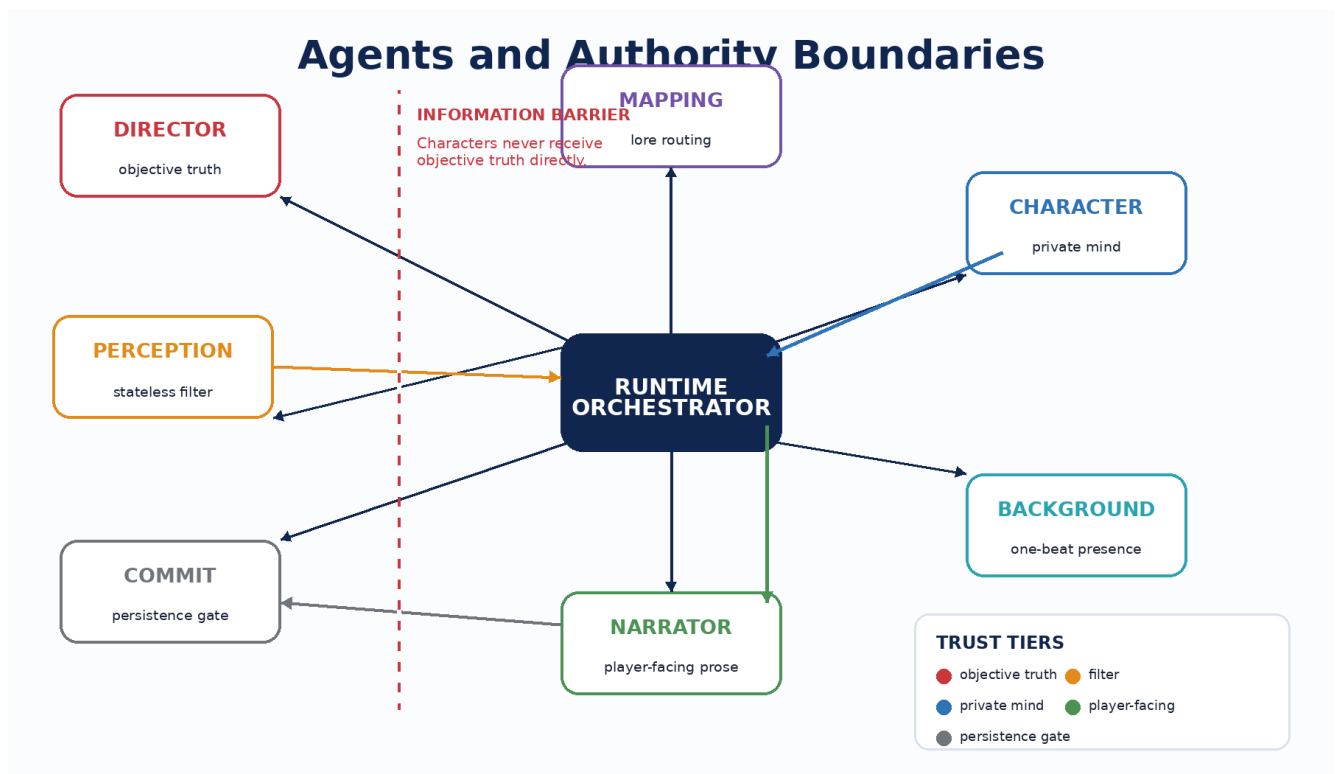


Figure 2. Runtime coordinates the system, while the information barrier keeps objective truth out of private character minds.

3. The Turn Lifecycle

Each turn runs through a PipelineContext object and is executed by agents/runtime.py. The runtime builds a plan dynamically from the interpreted player input, then dispatches only the stages required for that turn.

Normal turns

A normal turn begins by preserving and classifying the player's declaration. The engine then gathers lore, filters the action onset into observer-specific perceptions, obtains reactions or character declarations, resolves one objective event, computes outcome perceptions, renders prose, and commits the result.

Normal Turn Lifecycle

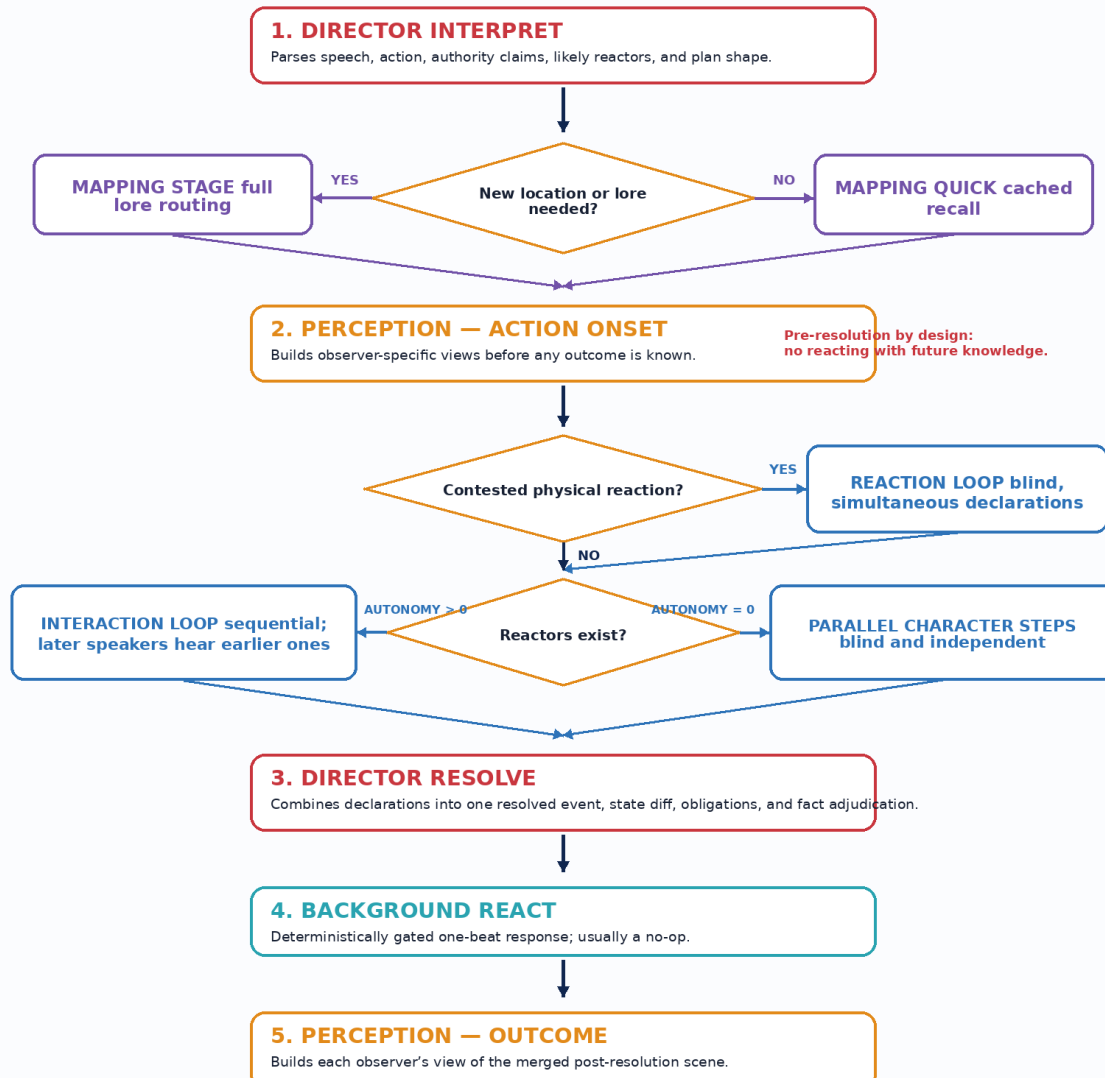


Figure 3. The normal turn separates what characters can react to from what the Director later resolves.

CRITICAL ORDERING

Perception of the action onset occurs before resolution. Characters therefore cannot react to the future result of an action they have not yet seen happen.

Opening turn

Turn zero uses a shorter establishment path. The world and initial actor state are created objectively, then filtered into the player's opening view and narrated.

Opening Turn (Turn 0)



The opening turn creates the world before presenting the first player-facing prose.

Figure 4. The opening turn establishes the world before producing the first player-facing scene.

4. Orchestration and Persistence

System	File	Responsibility
Runtime	agents/runtime.py	Builds the plan, dispatches stages, streams output, cancels work, resumes from a stage, rerolls one step, and reruns downstream stages.
Pipeline context	pipeline_context.py	Typed mutable turn state containing chat, turn, cast, player input, stage outputs, character results, and warnings.
Step storage	agents/storage.py	Stores one step per turn/key and immutable JSON variants with one active selection.
Commit	commit.py	Performs slow preparation before the lock, then writes all primary mutations inside one outer transaction.
Database	db.py	SQLite schema, migrations, connection pool, transaction helpers, and the world key/value store.
Checkpoints	checkpoints.py	Whole-chat snapshots per turn plus restore orchestration.

Inspectable variants

Every stage output is stored as an immutable variant. This means a turn can be inspected, edited, rerolled, or recomputed from a chosen stage without pretending the original output never existed.

Stored Steps, Active Variants, and Reruns

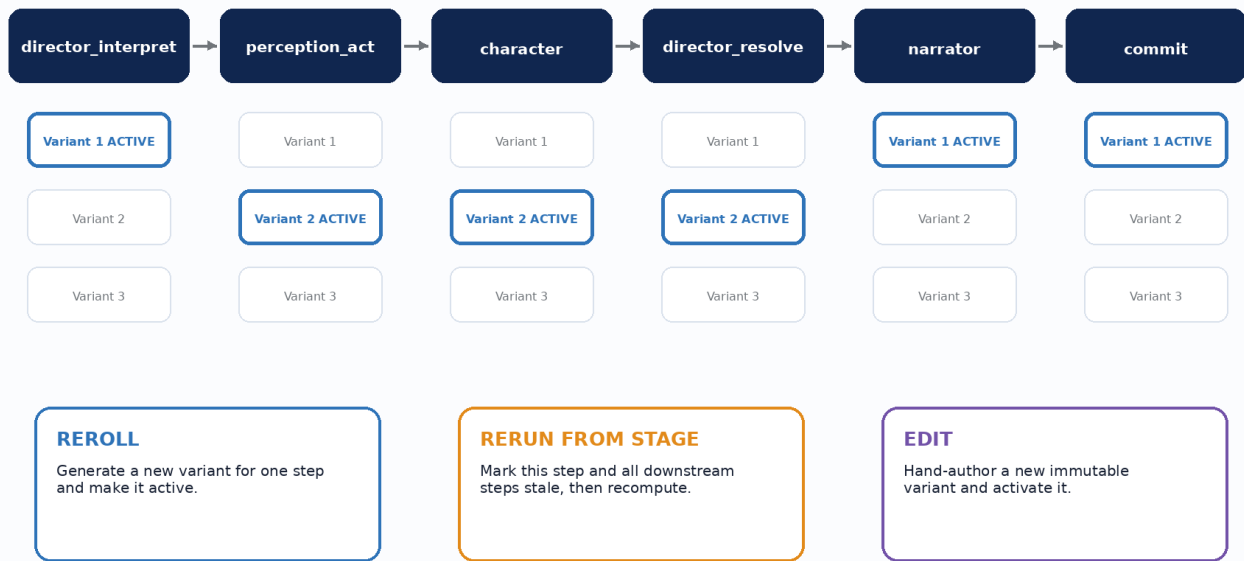
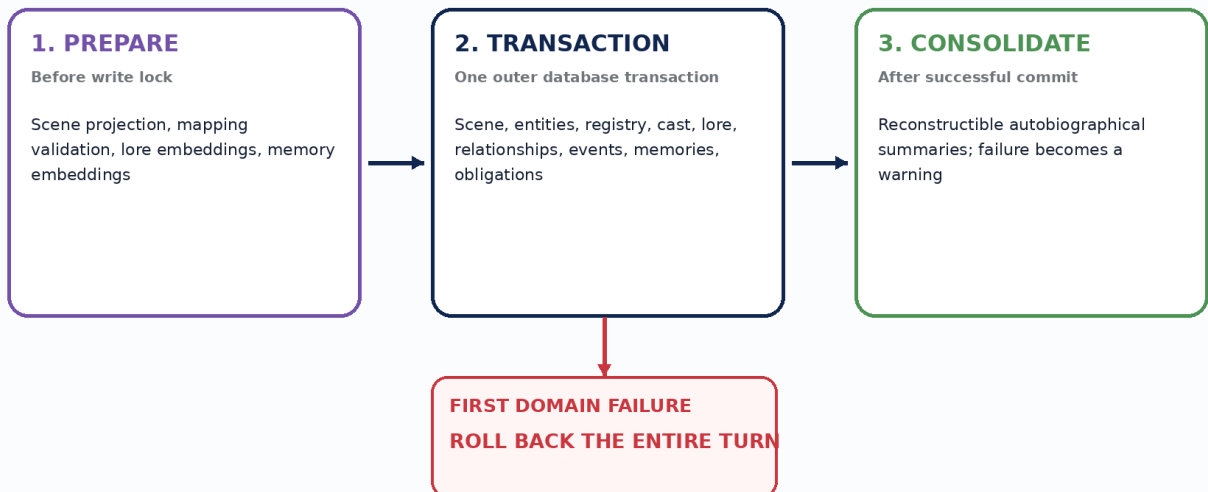


Figure 5. Each step can hold multiple immutable outputs, while one variant remains active.

Atomic commit

Upstream output remains provisional until `commit.py` validates and writes it. Slow work such as embeddings occurs before the write lock. All primary mutations then occur inside one transaction; a domain failure rolls the entire turn back.

Atomic Turn Persistence



Invariant: a turn commits all primary persistent effects, or none.

Figure 6. Primary state changes are all-or-nothing; only reconstructible summary work is allowed after the transaction.

PERSISTENCE INVARIANT

A turn either commits all primary persistent effects, or none.

5. The Character Mind

A character acts from a private payload containing identity, psychology, filtered perception, memory, relationships, models of other minds, and interior state. The Director sees the resulting declaration, not the private reasoning that produced it.

System	File	Responsibility
Character agent	agents/character.py	Builds the private payload and returns declared behavior.
Affect	affect.py	Computes goals, blended mood, drive strain, and rupture mechanics.
Theory of mind	theory_of_mind.py	Maintains fallible, confidence-limited models of other minds.
Memory	memory.py	Stores and retrieves episodic, semantic, dialogue, and inference memory plus relationships.
Schema	character_schema.py	Versioned character and persona sheets, normalization, and accessors.

Goals and earned change

- A stable core drive defines essence, expression, and taboo.
- Standing intentions persist across scenes, while per-beat wants track what the character is trying to enact or suppress.
- Sustained strain can open a drive-rupture window. Deterministic floors ensure that the window eventually resolves, closes, or produces a lasting shift.
- A completed shift leaves a scar in `former_drives`, preserving continuity rather than rewriting the character's past.

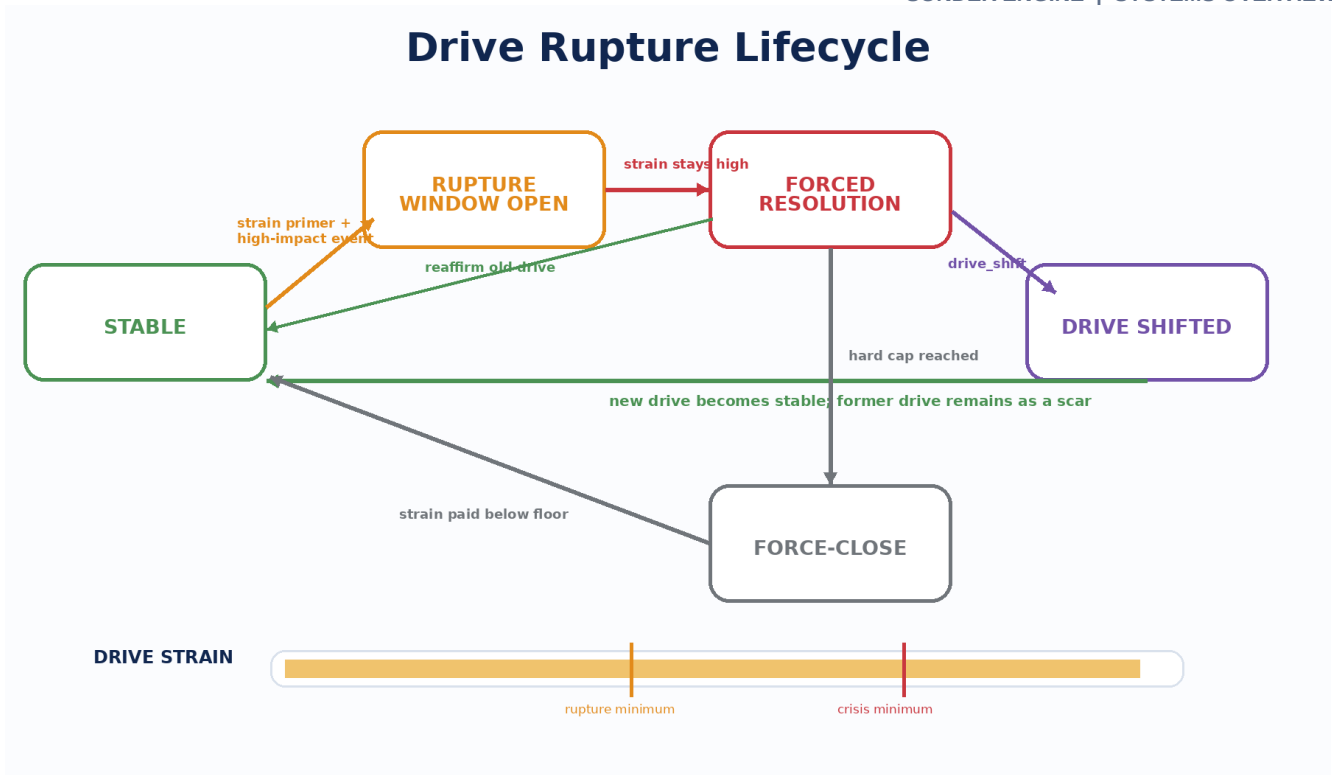


Figure 7. Drive rupture converts sustained internal strain into an earned, bounded opportunity for character change.

Blended affect

The model proposes goal impacts, but the engine reconciles them into canonical valence and arousal. Fast surface reactions sit over a slower emotional undercurrent and a stable temperament baseline, with deterministic decay between beats.

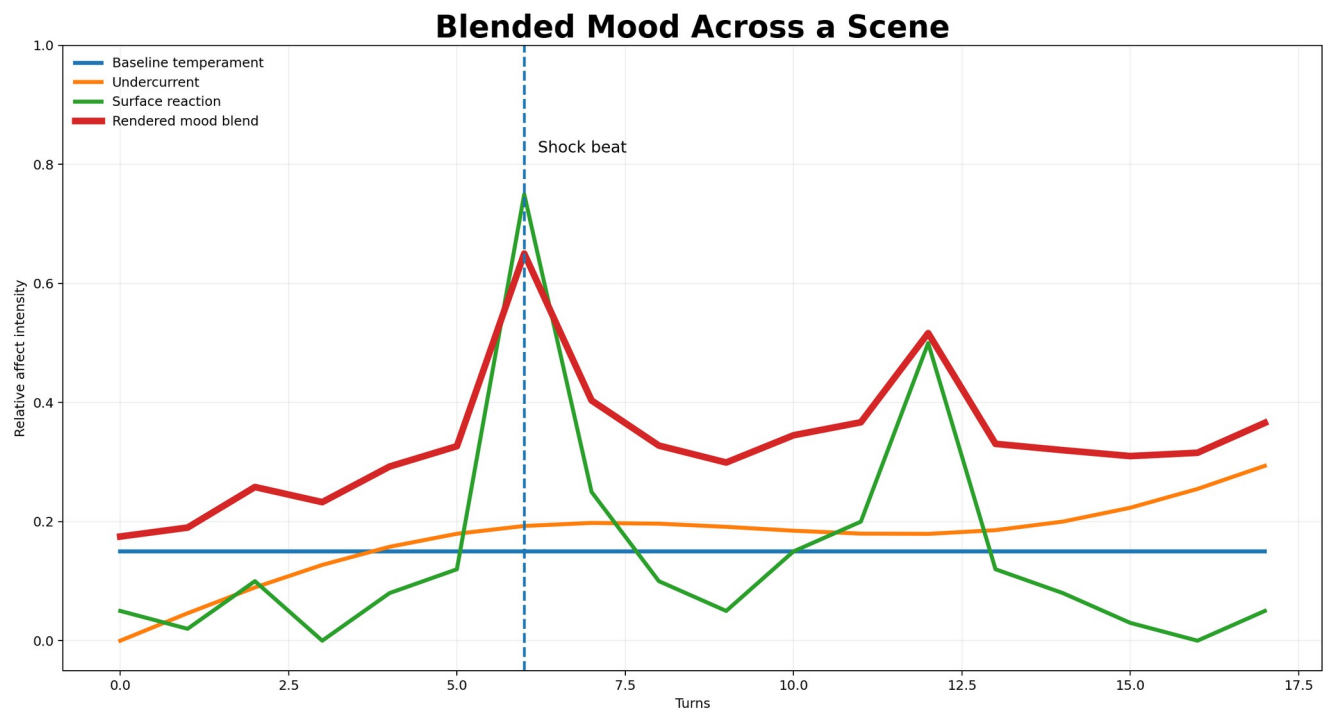


Figure 8. Surface reactions can spike while the slower undercurrent continues to accumulate across a scene.

6. Memory and Lore

memory.py serves two related retrieval systems: autobiographical memory for individual characters and the shared lore graph used by Mapping. They share vector-search machinery but obey different authority rules.

Character memory

Memory rows may be episodic, semantic, dialogue, or inference. Each record carries provenance such as witnessed, heard, told, inferred, or read; salience; emotional context; entities; location; and a vector embedding. Retrieval blends recency, salience, and semantic similarity to the current beat.

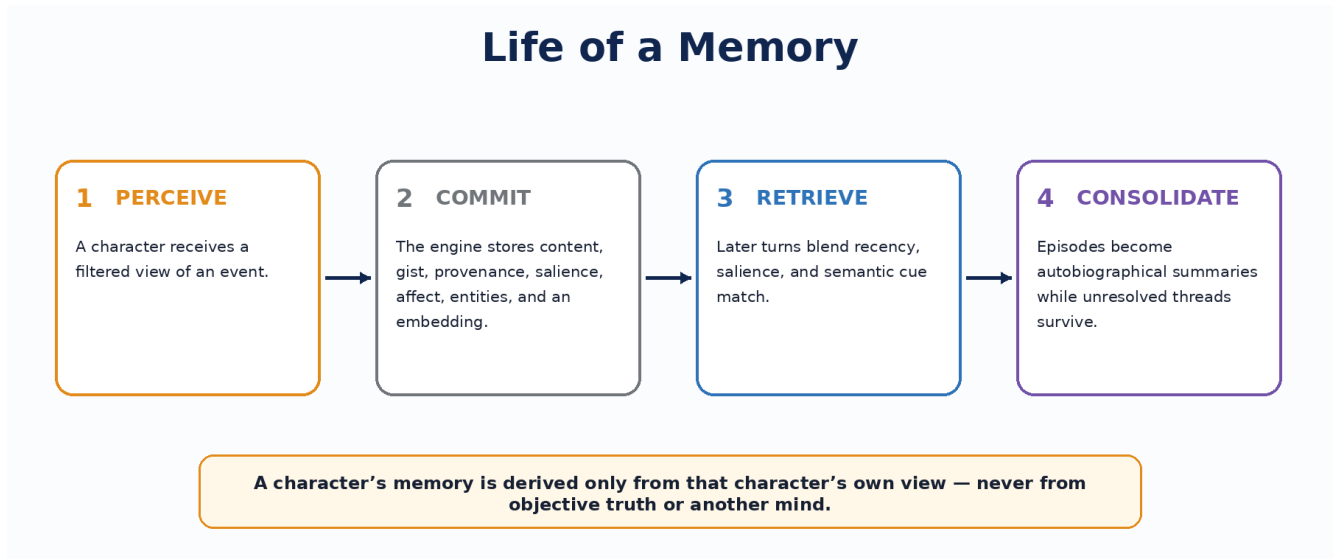


Figure 9. Memories originate from a character's filtered view and later consolidate into autobiographical summaries.

Lore graph

Lorebooks form a tree with typed cross-links. Each book contains searchable entries with keys, embeddings, and knowledge tags. Mapping decides which canon is relevant to the current scene, while retired regions remain searchable so the world retains history.

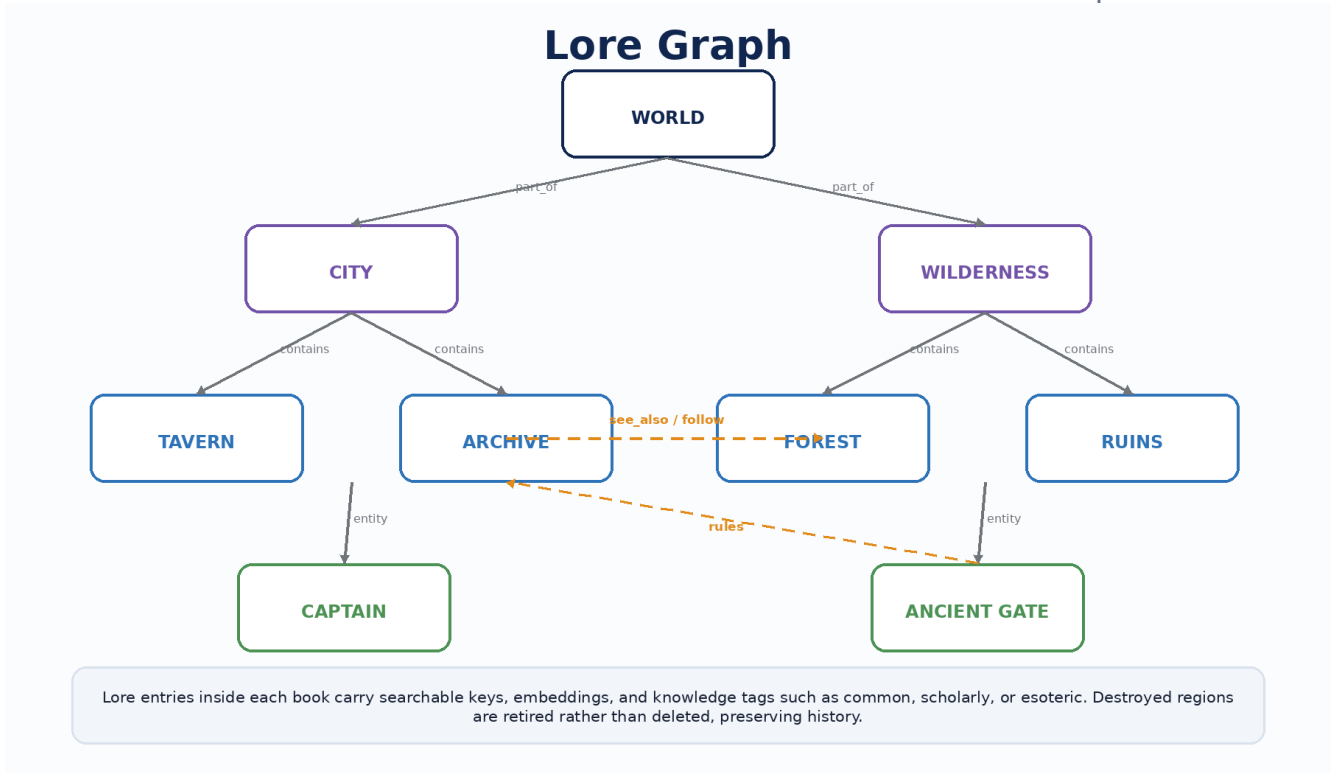


Figure 10. Lore is organized as nested books with typed links that Mapping can follow during retrieval.

7. World, Space, and Time

System	File	Responsibility
Scene	scene.py; world.scene	Authoritative live rooms, positions, and entity state for one temporal frame.
Spatial	spatial.py	Room adjacency, line of sight, and hearing attenuation; unknown relationships fail closed.
Egocentric space	spatial_frames.py	Per-character facing, bearings, left/right, came-from, and field of view.
Mechanics	mechanics.py	Commit-time sweep for due scheduled events, transit completion, and condition expiry.
Registry / entities	room_registry; world_entities	Cross-frame room identity and a derived projection of committed scene state.
Temporal frames	frames.py	Parallel timelines or eras, each with its own live world.
Paradox	paradox.py	Fixed-point and hazard handling for time travel and concurrent play.

Spatial access

Perception uses deterministic spatial facts. Open doors may grant sight and clear speech; closed doors attenuate sound; walls may reduce an event to a dull impact. Unknown relationships never grant accidental access.

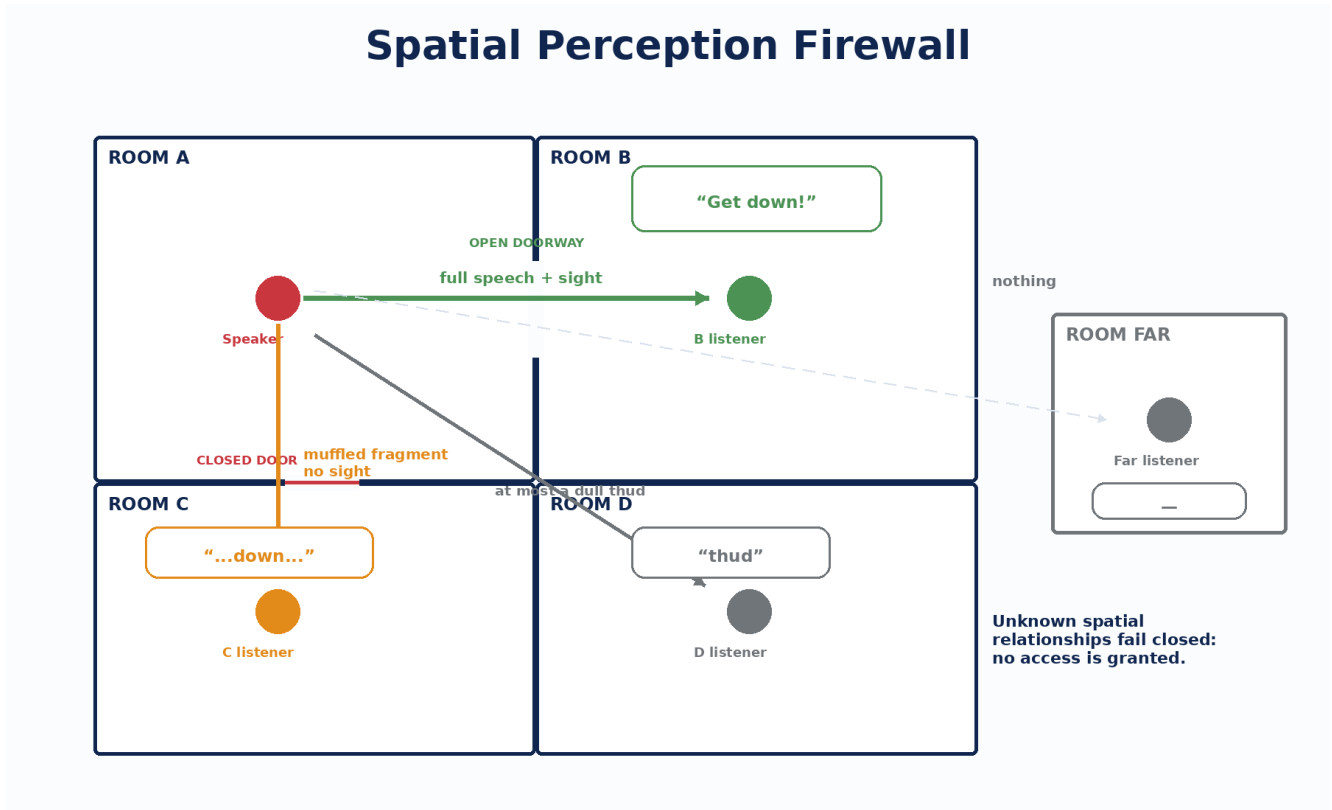


Figure 11. The same shout becomes different evidence for listeners in different rooms.

Temporal frames

A frame is a parallel live world inside one chat. A past-era frame can retain a building that has already been destroyed in the present. Cross-frame ledgers preserve room identity and lore, while paradox handling protects fixed points or flags causality hazards.

Temporal Frames and Paradox Handling

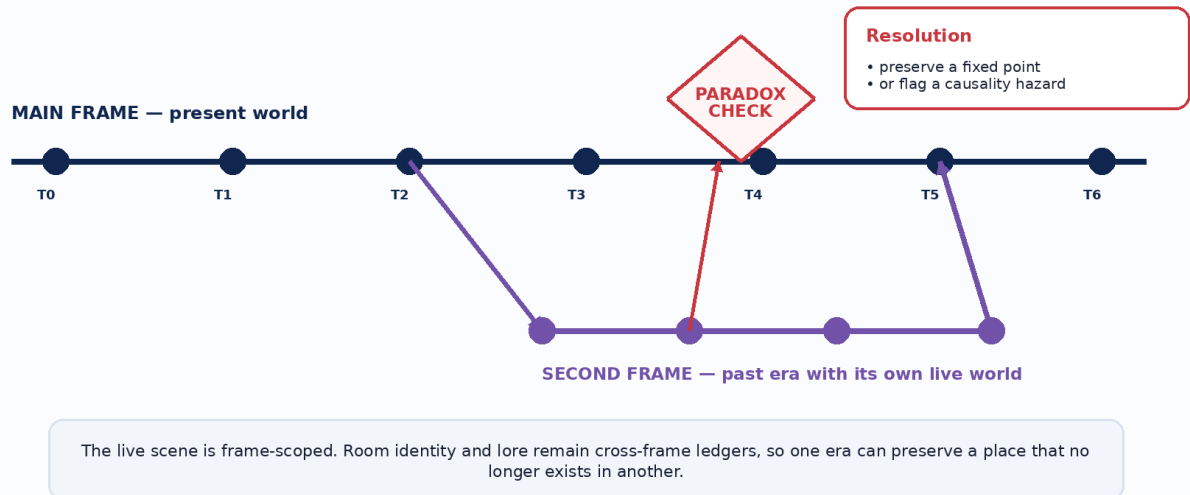


Figure 12. Frames can split and merge while carrying independent live scene state.

PHYSICAL TRUTH AUTHORITY

The frame-scoped world.scene blob is authoritative for live state. room_registry is the cross-frame identity ledger; world_entities is a derived projection.

8. Providers, Contracts, and Input/Output

System	File	Responsibility
Providers	providers.py	Per-role model selection, retries, streaming, cancellation, embeddings, provider backends, and prompt caching.
Output contracts	schemas.py	Pydantic contracts and semantic validation for every agent payload.
JSON hygiene	llm_quality.py	Strict parsing, schema validation, and model-assisted repair.
Prompts	prompts.py	Default role instructions and preset access.
Import / generate	importers.py; greetings.py	Character, persona, and lorebook import plus story initialization from greetings.
API and UI	app.py; static/js/*	FastAPI resources and turn control, server-sent event streaming, and browser frontend.
Access control	guest_access.py	Host sessions, single-use join codes, scoped guest tokens, and deny-by-default behavior.

System	File	Responsibility
Self-update	updates.py	Fast-forward-only update from the configured Git origin.

The `agent_models` configuration can route every role to a different provider and model tier. Expensive reasoning can be reserved for high-authority roles, while background or utility work can use faster models. Explicit fallbacks improve reliability without hiding missing configuration.

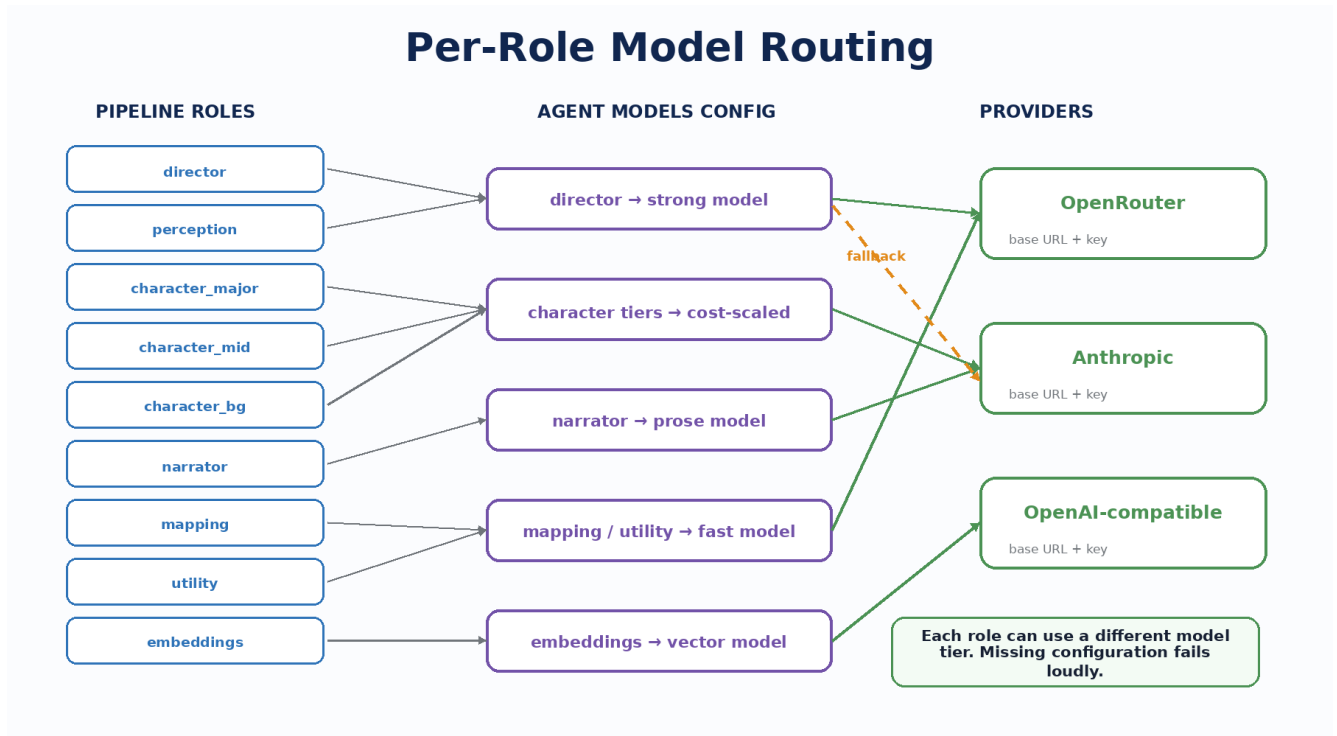


Figure 13. Roles are independently routed through configuration to provider/model combinations and fallbacks.

9. The Data Model

The relational backbone is intentionally small. Rich state primarily lives in JSON: the world key/value store, character sheets, frame-scoped scene blobs, and immutable variant contents.

The turn spine is `chats → turns → steps → variants`. Cast membership is represented by characters and `chat_chars`, with frame-specific status where needed. Memories, lore, world state, frames, and checkpoints attach to the chat around that spine.

Core Data Model

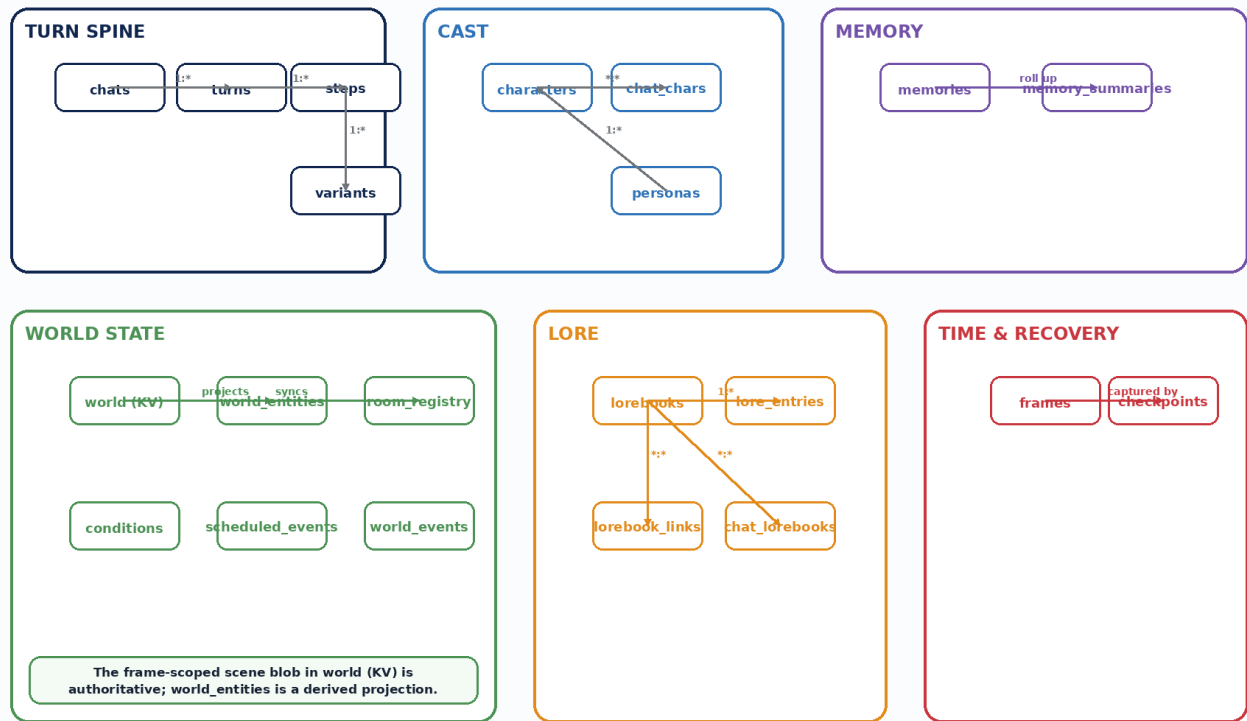


Figure 14. The schema is grouped around the turn spine, with supporting clusters for cast, memory, world state, lore, time, and recovery.

SCHEMA PRINCIPLE

The database stores enough structure to validate, rerun, recover, and query the story without forcing every rich fictional state into rigid relational columns.

10. Cross-Cutting Design Principles

EPISTEMIC LEAST PRIVILEGE

Every role receives the minimum context required for its job. The Narrator does not see private minds; a Character does not see objective truth.

CAPTURE, DO NOT GATE

Interpretation captures everything the player declared, including asserted world facts. Validation occurs later rather than silently discarding input.

PERSISTENCE IS EARNED

Model output is provisional until commit validates it. A background presence gains memory only after promotion into a persistent character.

STRUCTURE OVER INSTRUCTION

The engine relies on typed payloads, deterministic filters, validation, and behavioral floors rather than hoping one large prompt is obeyed.

AUDITABILITY AND RECOVERY

Every stage is stored as a variant, every committed turn can be checkpointed, and retired world regions remain historically retrievable.

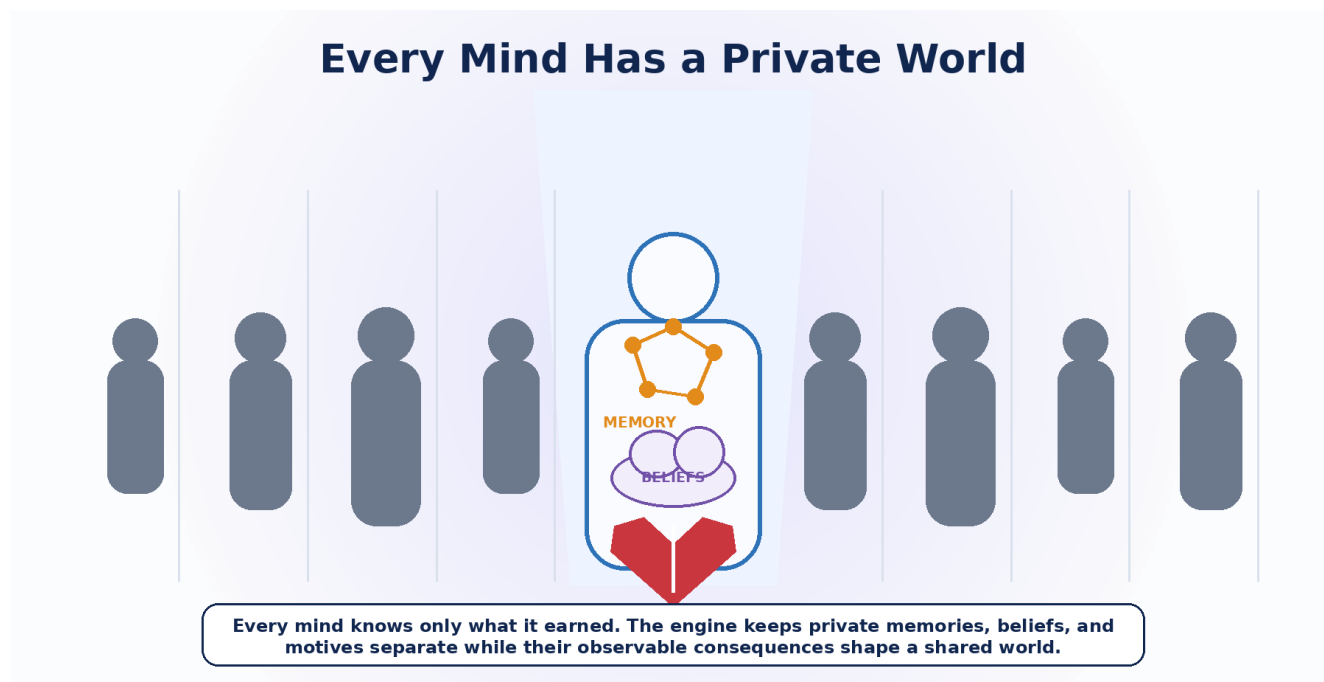


Figure 15. The architecture exists to preserve separate inner lives while allowing their observable consequences to shape a shared world.

Appendix A. Code Navigation

Question	Start here
Where is the exact execution order?	PIPELINE.md
Where is the module, table, and route index?	CODE_MAP.md
How do schema changes and write helpers work?	DATABASE.md
Where are the architecture philosophy and roadmap?	Design.md
Where is the current fix backlog?	AUDIT_FOLLOWUPS.md
Where are edit-routing rules and invariants?	AGENTS.md
How is a new pipeline stage added?	agents/README.md

This overview is intended as the friendly orientation layer. The linked project documents remain the authoritative references for implementation details, exact execution order, schema migrations, and current known weaknesses.